Bitcask Explained: A Log-Structured Fast Key/Value Store

Architectural breakdown based on the engineering dissection by **Arpit Bhayani** (*Asli Engineering*, Video ID: `OWI_tJRUMqM`) and the foundational paper by Justin Sheehy and David Smith (Basho Technologies).

> **The Fundamental Premise of Bitcask**
> Traditional disk-based storage engines rely on in-place updates via B-Trees, incurring heavy random I/O and locking contention. Bitcask flips this paradigm by combining **pure append-only sequential disk writes** with an **in-memory hash index (KeyDir)** containing the byte offsets of every active key. This guarantees exactly **one sequential write per mutate** and at most **one disk seek per read**.

1. The Problem Space: Why B-Trees & Random Writes Fail at Scale

Storage hardware (both spinning disks and flash NAND SSDs) delivers orders of magnitude higher throughput for sequential I/O compared to random block modifications. Bitcask was engineered at Basho (and notably leveraged in high-throughput production at companies like Uber) to satisfy distinct engineering constraints:

Random I/O Bottlenecks (B-Trees) • Write Amplification

- In-place page updates require random seeks across data blocks.
- Page splits cause double-buffering and heavy write amplification.
- Concurrency demands complex node-level page latches and WAL locks.
- Crash recovery requires parsing and replaying large WAL logs.

Bitcask Design Objectives • Append-Only Architecture

- **Ultra-low latency:** Exactly 1 disk seek for any random read.
- **High write throughput:** Always sequential disk appends.
- **Crash friendliness:** Zero recovery corruption; partial writes discarded.
- **Single OS process writer:** Lockless, race-free append coordination.

2. Storage Engine Architecture Comparison

Storage Engine	Write Path Mechanics	Read Path Mechanics	Primary Trade-Off
Bitcask	Direct sequential append to active log; in-memory KeyDir updated.	O(1) in-memory hash lookup → exactly 1 direct disk seek.	**RAM bound:** All keys must permanently fit in memory. Range queries unsupported.
LSM-Tree (RocksDB)	Append to WAL + in-memory MemTable; tiered SSTable compaction.	MemTable → Bloom Filters → multiple SSTable levels.	Higher read amplification (multiple file reads); complex multi-level compactions.
B+ Tree (PostgreSQL, InnoDB)	Random write to memory buffer pool + sequential append to WAL.	Tree traversal (root-to-leaf); page hits cache or seeks disk.	Heavy write amplification; page splits and random dirty page flushes.

3. The Dual-Plane Architecture: Append Log & In-Memory KeyDir

A Bitcask instance resides inside an OS directory containing multiple segmented files. Exactly one file is the **Active File** (open for writing); all past segments are marked **Immutable** (read-only).



Figure 1: The Bitcask architecture: RAM-based KeyDir maps key names to on-disk segment offsets and payload sizes.

On-Disk Record Layout

Each appended record contains a fixed 16-byte prefix followed by variable fields:

- `crc (4 bytes)`: IEEE 802.3 checksum verifying data integrity.
- `tstamp (4 bytes)`: 32-bit Unix epoch timestamp.
- `ksz (4 bytes)` & `value_sz (4 bytes)`: Explicit lengths.
- `key` and `value`: Raw binary payloads.

KeyDir Hash Entry Anatomy

The in-memory Hash Table stores lightweight pointers:

- `file_id`: ID of the data file storing the active record.
- `value_pos`: Absolute byte offset of the record start.
- `value_sz`: Exact byte size of the value payload.
- `tstamp`: Used for compaction validation and dispute resolution.

4. Core Execution Workflows: Write, Read, and Delete Paths

Bitcask eliminates complicated multi-stage transaction managers by enforcing strict append-only constraints:

Operational Path Mechanics

1. Put(K, V): Serialize binary entry → append to active file log → update `KeyDir[K]` with new `{file_id, offset, size, tstamp}`. Once the active file reaches size threshold (e.g., 2 GB), it closes and becomes immutable.

2. Get(K): Lookup `KeyDir[K]`. If absent, return `nil`. If present, invoke a single `pread()` at `(file_id, offset)`, verify the CRC, and return the value.

3. Delete(K): Appends a record with a special **Tombstone marker** in place of value. KeyDir removes the key entry immediately from RAM.

5. Background Compaction (Merging) and the Hint File Innovation

Because updates and deletions do not overwrite data in-place, stale records and tombstones accumulate on disk. Bitcask runs an asynchronous **Merge Process** across immutable segments to reclaim space.

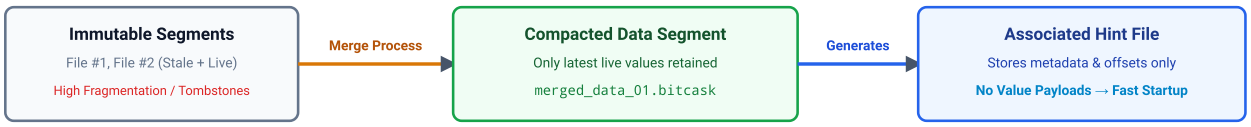


Figure 2: Compaction merges fragmented files and generates lightweight Hint files to enable fast startup recovery.

The Startup Bootstrapping Problem

Upon server crash or restart, the in-memory `KeyDir` is lost. Without hints, Bitcask must sequentially scan every gigabyte of raw data files to rebuild the hash index, leading to protracted multi-minute recovery times.

How Hint Files Solve Recovery

Compaction outputs twin files: a compacted `.data` file and a tiny `.hint` file. The hint file stores identical headers but **omits the value payloads**. On restart, the engine only reads the small hint files, reconstructing KeyDir in seconds.

6. Crash Guarantees & Production Trade-Offs

Engineering Analysis: When to Use (and When to Avoid) Bitcask

Crash Resilience: Writes are atomic appends. If a crash interrupts a write, the trailing corrupted bytes fail the CRC32 check and are cleanly truncated at recovery without compromising historical records.

Ideal Workloads: High-volume key-value stores where the key set fits comfortably in RAM (e.g., driver location pointers, user session states, fast caching tiers) requiring sub-millisecond p99 latencies.

Disqualifying Constraints: If key cardinality exceeds physical RAM, or if range queries (e.g., `scan(key_a..key_z)`) are needed, Bitcask is unsuitable. An LSM-Tree or B+ Tree must be selected instead.